Sentrix Whitepaper

Sentrix

A Layer-One Blockchain for the Real Economy

Author: Satya Kwok **Version:** 1.2 (final unless chain hard-fork)

Abstract

We propose Sentrix, a Layer-One blockchain optimized for real-world economic settlement. Sentrix uses a delegated-proof-of-stake validator selection paired with a three-phase Byzantine Fault Tolerant agreement protocol to produce one-second-final blocks. Native protocol operations—token issuance, staking, validator coordination—execute directly against canonical state, while a second execution rail runs the Ethereum Virtual Machine for general-purpose programmability. Monetary policy is fixed: a maximum supply of 315 million SRX, a one-SRX block reward halving every approximately 126 million blocks (four years at one-second blocks), and a fee mechanism that destroys 50% of every transaction fee forever. The chain is designed for durable financial infrastructure for real-world economic activity, beginning in Indonesia and scaling outward. This paper specifies the design rationale, architecture, transaction lifecycle, consensus safety properties, monetary mechanics, and threat model.

---

1. Introduction

Most public blockchains were designed to do something other than serve real economic activity. Bitcoin was designed to be a peer-to-peer digital cash system [1], but in practice operates primarily as a settlement layer for value storage. Ethereum was designed as a world computer [2], but its on-chain economy is dominated by financial speculation, on-chain trading, and synthetic assets. Layer-Two scaling solutions inherit these orientations and amplify them.

The disconnect between blockchain infrastructure and the actual economic activity of the majority of the world's population is striking. The bulk of human commerce remains denominated in local currencies, settled through banking rails that have not materially improved in decades, and effectively excluded from public-blockchain participation. The friction is not philosophical—real businesses, real cooperatives, real cross-border merchants want fast and final settlement—but architectural. The infrastructure they would need is not the infrastructure that has been built.

Sentrix is a response to this architectural gap. It is a blockchain whose every design choice prioritizes real economic settlement over trading speculation. Where general-purpose chains favor maximum flexibility at the cost of efficiency, Sentrix promotes the most common economic primitives to native protocol operations, eliminating an entire class of overhead. Where most chains tolerate ten- or thirty-second block times, Sentrix targets one-second finality, sufficient for retail-grade interactions. Where many chains follow inflationary token

models, Sentrix is hard-capped, halving, and deflationary—every fee is partially destroyed forever, so circulating supply contracts as activity grows.

Sentrix is built first for Indonesia, where the population, unbanked share, and remittance volume present a uniquely large unmet demand. From there it is intended to scale outward.

---

2. Vision, Mission, and Rationale

2.1 Vision

A future in which real-world economic activity—cross-border payments, asset-backed lending, retail commerce, cooperative savings, supply-chain settlement, identity verification—runs on transparent, low-cost, programmable rails owned by no single corporation, accessible from any internet connection, and as fast as a credit-card swipe.

We do not believe this vision is achievable through retrofits of legacy banking systems. We do not believe it is achievable on blockchains designed for trading speculation. It is achievable through purpose-built infrastructure that treats real-economy settlement as a first-class concern rather than an afterthought.

Sentrix is one implementation of that infrastructure.

2.2 Mission

To build the most reliable, low-cost, and accessible settlement layer for real-world economic activity, beginning with the geographies and use cases that legacy infrastructure has failed to serve.

Concretely, this means:

- One-second final settlement for any transfer, swap, or contract call.
- Transaction costs measured in fractions of a cent, regardless of transfer size.
- Native protocol support for token issuance, staking, and asset management without requiring smart-contract deployment.
- Compatibility with the global Ethereum developer toolchain so existing applications port without modification.
- Open codebase, transparent operations, deterministic monetary policy.
- A market-entry orientation toward Indonesia and Southeast Asia, where unmet demand is largest.

2.3 Why Sentrix Exists

The infrastructure problem we address has four dimensions:

Latency. Existing payment rails settle in days (correspondent banking), hours (card networks), or tens of seconds (blockchains optimized for security or trading). Real commerce demands sub-second confirmation for point-of-sale, retail, and routine business operations.

Cost. Cross-border remittance fees of 5–10% remain common. Card-network interchange fees of 1–3% extract value at every retail transaction. Smart-contract gas costs on general-purpose chains routinely exceed the value being transferred for small payments. Sentrix’s native operations target costs measured in fractions of a US cent regardless of the underlying asset value.

Programmability without overhead. Blockchains have demonstrated that programmable transactions enable powerful new economic constructions—lending, escrow, automated market making, identity verification. But routine operations such as transferring a token, claiming staking rewards, or registering a validator should not require deploying audited smart-contract code. They should be part of the protocol.

Inclusion. Banking infrastructure correlates with national wealth. Public blockchain infrastructure could in principle break that correlation, but in practice has reproduced it: the vast majority of on-chain activity originates from a small set of high-income jurisdictions. Sentrix is positioned to invert this default by building first for the geographies the legacy system serves least well.

2.4 Why Now

Three preconditions have converged that make this work feasible.

First, **the EVM has matured into a standard.** Tooling, wallets, smart-contract libraries, security audit conventions, and developer mental models have converged. A new chain that adopts EVM compatibility inherits the entire ecosystem at zero cost.

Second, **Byzantine Fault Tolerant consensus has been productionized.** Tendermint-style BFT has run reliable mainnets for years. The hard parts of consensus are now well-trodden, and a new chain can build on robust open-source implementations.

Third, **Rust has matured into a viable systems language for blockchain implementation.** Memory safety guarantees, mature async runtimes, performant cryptographic libraries, and a culture of zero-copy efficiency make solo or small-team chain development feasible in a way it was not a decade ago.

A chain that combines these three—EVM compatibility, BFT consensus, and a Rust implementation—and applies them to the real-economy use case is technically feasible today in a way it was not feasible five years ago.

2.5 Why Indonesia First

Indonesia is the world’s fourth-most-populous country and Southeast Asia’s largest economy. It has:

- A young, mobile-first population with high smartphone penetration.
- A large unbanked or underbanked population (~40% by some estimates).
- Strong informal economic structures (cooperative banking, community lending, microfinance) that map naturally to blockchain settlement.
- A growing remittance flow at the scale of tens of billions of dollars annually.
- A regulatory environment that, while still maturing, has shown willingness to engage with crypto and blockchain.
- A meaningful local crypto-aware developer community.

The combination of large unmet demand, technological readiness, and cultural fit makes Indonesia a natural starting point. Once a settlement layer is durable and useful in this market, expansion to comparable markets in Southeast Asia and beyond becomes straightforward.

This is not “emerging markets” framing. It is specific-market framing. Sentrix is built to serve a particular set of users with particular needs, and is designed to grow outward from there.

3. Design Principles

Six principles shape every architectural decision in Sentrix:

1. Settlement over speculation. A blockchain optimized for trading creates incentive structures that produce trading. A blockchain optimized for settlement produces commerce. We choose the latter at every fork in the road.

2. Native primitives over contract overhead. Operations that almost every user will perform—holding tokens, staking, claiming rewards, transferring assets—should not require deploying or interacting with smart contracts. They are part of the protocol.

3. EVM compatibility for the general case. Where programmability is required, it is well served by the Ethereum Virtual Machine, an evolving standard with broad tooling support. Sentrix runs the EVM faithfully alongside its native operations.

4. Deflationary monetary policy. Inflationary tokens reward early holders at the expense of late ones. Sentrix is supply-capped, halving on a Bitcoin-parity schedule, and burns half of every transaction fee. As activity grows, supply contracts.

5. Crypto-economic security through staking. Decentralization is a property of who can stake, not who currently does. Sentrix's validator set is open, permissionless, and economically secured by stake at risk.

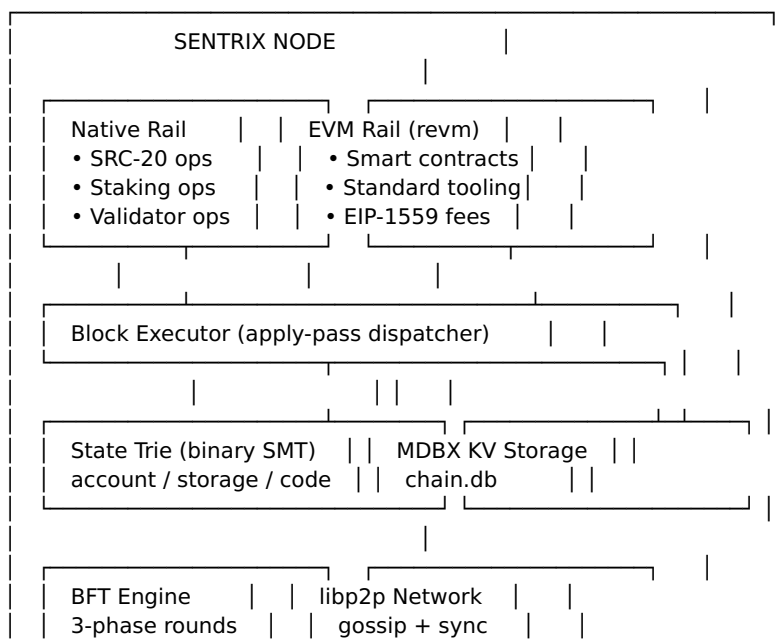
6. Real over nominal. Real-world assets, real-world businesses, real economic activity. Sentrix avoids on-chain abstractions whose value derives only from other on-chain abstractions. The chain serves the world; the world does not serve the chain.

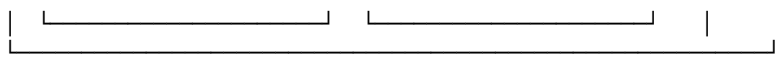
---

4. Architecture

Sentrix is structured as four integrated subsystems operating on a single canonical state.

Figure 1 — System architecture





4.1 Consensus Layer

Sentrix uses a delegated-proof-of-stake selection model paired with a three-phase Byzantine Fault Tolerant agreement protocol [3].

4.1.1 Validator Selection

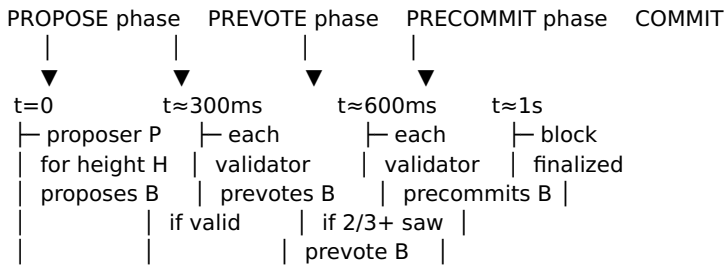
The validator set is selected by stake-weighted delegation. Any token holder may delegate to any validator candidate; the resulting weighted ranking determines the active set for each epoch. Active-set membership is recomputed at fixed epoch boundaries, allowing new validators to enter and underperforming validators to exit without governance intervention.

The minimum self-stake for a validator is enforced at the protocol level. Delegated stake is bonded with an unbonding period: stake withdrawal initiates a fixed-duration timeout during which the stake remains slashable but is not eligible for new rewards. This prevents validators from offloading risk immediately before misbehaving.

4.1.2 Three-Phase Round

Within an epoch, the active set runs a Tendermint-style three-phase round to finalize each block. A round consists of three message types and three corresponding phase transitions:

Figure 2 — BFT round timing



A block is final once a supermajority (≥⅔ of stake-weighted active set) precommits the same block in the same round. The justification—the set of precommit signatures—is included in the next block, providing public proof that the parent was finalized.

4.1.3 Locking Rules and Round Skip

Once a validator has prevoted a block in a round, it locks on that block. In subsequent rounds at the same height, it will only prevote a different block if it receives a “polka” (≥⅔ prevotes for the new block) at a higher round. This locking property is what guarantees safety: two conflicting blocks cannot both gather supermajority precommits.

If a round fails to finalize within timeout (proposer offline, network partition, validator absence), the protocol advances to the next round. The proposer rotates round-robin through the active set. After a configurable number of consecutive failed rounds, the round timeout doubles, providing weak-synchrony recovery.

4.1.4 Safety and Liveness

Under the standard BFT assumption that fewer than one-third of stake-weighted active validators are Byzantine, two safety theorems hold:

- **Agreement:** No two honest validators finalize different blocks at the same height.
- **Validity:** A finalized block is well-formed and applies cleanly.

Liveness holds under a weak-synchrony assumption: messages between honest validators eventually deliver, possibly with bounded delay. Under this assumption, the protocol guarantees that progress eventually occurs.

If the assumption is violated (≥⅓ Byzantine power), safety is no longer guaranteed and the chain may fork. Recovery in this case requires social-coordination mechanisms—identifying the canonical chain through external trust signals—as in any BFT system.

4.2 Execution Layer

Two execution rails operate on the same canonical state.

Ethereum Virtual Machine. Sentrix runs the EVM through a high-performance Rust implementation [4]. Standard Ethereum contracts (ERC-20, ERC-721, Uniswap-style pools, lending protocols) deploy without modification. Standard tooling—Foundry, Hardhat, MetaMask, ethers and viem—works directly. Gas accounting follows the EIP-1559 model: a per-block base fee that adjusts to demand, plus a sender-set tip that pays the proposer for inclusion.

Native protocol operations. Token issuance (SRC-20), staking (delegate, undelegate, claim rewards), and validator coordination are not smart contracts but transactions interpreted directly by the protocol. They cost less gas than contract-equivalent operations because they bypass the EVM entirely. They cannot be exploited through unaudited code because their behavior is fixed at the protocol level.

The two rails interoperate. A user holding SRC-20 native tokens can interact with EVM contracts that read those balances through a canonical balance-query precompile. EVM contracts can authorize native operations through a system-call gateway. The chain's canonical state is a single merge of EVM state and native ledger.

4.3 State Layer

Sentrix maintains a binary sparse Merkle tree as the canonical state representation [9]. State roots are stamped into each block past a defined activation height, providing the cryptographic guarantee that any node which has applied the same blocks reaches the same state. Divergent state roots produce divergent block hashes, and BFT ensures the chain converges on a single canonical history.

State is persisted in MDBX [10], an embedded key-value store optimized for high-throughput random reads. The full state is local to every full node; light clients can verify against the trie root with logarithmic Merkle proofs.

4.3.1 State Transition Function

A Sentrix block applies a sequence of transactions to a prior state to produce a new state. Formally, the state transition function APPLY(S, B) takes a state S and a block B = (header, txs) and produces either a new state S′ or INVALID:

```
APPLY(S, B):
  1. Verify B's header (parent hash, timestamp, proposer signature).
  2. For each transaction TX in B.txs, in order:
     a. Verify TX signature against TX.from.
     b. Verify TX.nonce == S[TX.from].nonce.
     c. Compute fee = TX.gas_used × TX.gas_price (EVM rail) or
```

MIN_TX_FEE (native rail).
d. If S[TX.from].balance < fee + TX.value: skip (insufficient).
e. Deduct fee from TX.from.
f. Burn 50% of fee to BURN_ADDRESS; pay 50% to block proposer.
g. Apply TX:
- EVM rail: invoke revm with TX as call into TX.to.
- Native rail: dispatch op_type to native handler.
h. If apply succeeds: update accounts, increment nonce, emit events.
If apply fails: revert account state, charge fee anyway.
3. Compute new state root from updated trie.
4. Verify new state root matches B.header.state_root.
5. Return S′ if all checks pass; else INVALID.

The function is deterministic: every node executing the same `S` and `B` produces bit-identical `S′`. This determinism is the property that allows independent nodes to verify each other's claims about chain state.

4.3.2 Light Clients

A light client does not store full state. It tracks block headers and verifies specific facts by requesting Merkle proofs from full nodes. Given a block header containing the canonical state root, a light client can verify any account balance, contract storage slot, or transaction inclusion using a logarithmic-size proof against the root. This makes mobile wallet usage, browser-based dApps, and constrained-resource integrations practical without compromising security: the light client trusts cryptographic proofs against a state root it has independently verified, not the responding node itself.

4.4 Network Model

Sentrix nodes communicate over a libp2p [11] mesh. The protocol uses three message classes:

- **Block gossip:** Newly finalized blocks propagate through gossipsub with topic `sentrix/blocks/1`. Receivers verify the block locally and apply it to canonical state.
- **Transaction gossip:** User-submitted transactions propagate through topic `sentrix/txs/1` until a validator includes them in a proposed block.
- **BFT messages:** Proposals, prevotes, and precommits are direct request-response between validators on topic `sentrix/bft/1`. Rebroadcast amplifies messages that may have been dropped.

Peer discovery uses a Kademlia DHT [12] with seed peer addresses for bootstrap. Each node also publishes a "validator advertisement" record containing its current libp2p multiaddrs, allowing other validators to maintain direct connections without external coordination.

A node joining the network bootstraps as follows:

1. Connect to one or more seed peers (configured at startup).
2. Run Kademlia DHT walk to populate peer routing table.
3. Request the chain head from peers; identify the longest stake-weighted chain.
4. Sync block-by-block from a peer until current head is reached.
5. Subscribe to gossip topics; begin participating in BFT if a validator.

The network's failure mode under partition is well-defined: a minority partition halts (cannot reach supermajority); a majority partition continues with reduced active set. Once the partition heals, the minority detects the longer canonical chain and rejoins.

4.5 Transaction Format

A Sentrix transaction has the following structure:

```
Transaction {
   chain_id:  uint16,       // 7119 mainnet, 7120 testnet
   nonce:     uint64,       // sender's account sequence
   op_type:   uint8,        // 0=native transfer, 1=SRC-20, 2=staking, 3=EVM, etc.
   from:      address20,    // 0x-prefixed 20-byte hex
   to:        address20,
   value:     uint64,       // amount in sentri (1 SRX = 10^8 sentri) for native
                            // or wei (1 SRX = 10^18 wei) for EVM
   gas_limit: uint64,       // EVM rail only; native ops use MIN_TX_FEE
   gas_price: uint64,       // wei per gas; EVM rail only
   data:      bytes,        // EVM call data, or native op payload
   signature: secp256k1,    // 65 bytes (r, s, v)
}
```

The `op_type` discriminator identifies whether the transaction targets the native rail or the EVM rail. The signature is verified against the canonical EIP-155 signing payload, providing replay protection across chains. Maximum transaction size is configured at the protocol level; oversize transactions are rejected by the mempool.

---

5. Native Operations

A central design choice in Sentrix is the promotion of common economic primitives from smart contracts to protocol operations. We expand on the reasoning here because it differs from the dominant approach in contemporary chains.

5.1 The Native vs Contract Question

The standard pattern in EVM chains is to implement everything—including the most common operations like token transfers and staking—as smart contracts. The abstraction is uniform: all operations cost gas, all are subject to contract security audits, all are programmable. This is conceptually clean but incurs three costs.

First, **execution overhead**. A simple token transfer in EVM space requires loading contract bytecode, dispatching to a function selector, performing storage reads and writes through the contract's namespace, and emitting events. The minimum cost is approximately 21,000 gas plus contract execution. The actual computation—decrement sender balance, increment receiver balance—is two storage operations.

Second, **security overhead**. Every contract-implemented primitive must be independently audited. Token contracts have produced billions of dollars in losses through bugs in long-deployed code. Native operations are part of the chain's consensus, audited once, deterministic forever.

Third, **upgradability friction**. Contract-implemented primitives are difficult to upgrade without coordinated migration. Native primitives evolve through hard forks, where the entire network upgrades atomically.

5.2 Native Operation Set

Sentrix's native operations include:

- **SRC-20 Token Operations.** Issue a fungible token, transfer it, approve allowances. Implemented as transaction variants directly applied to the native ledger. Token state lives in the canonical state trie alongside account balances.

- **Staking Operations.** Delegate stake to a validator, undelegate (with unbonding period), claim accrued rewards, register as validator candidate. Applied directly to the stake registry.

- **Native Transfer.** The simplest case: move SRX between accounts. Costs the protocol minimum fee (MIN_TX_FEE = 10,000 sentri = 0.0001 SRX). 50% burned, 50% to proposer.

- **Validator Coordination.** Activate, deactivate, and rotate validators through stake-weighted selection. No governance contract; the protocol applies the rotation each epoch.

These operations remain fully programmable: an EVM contract can call them through a system gateway, and any wallet that supports Sentrix can execute them through a single signed transaction. The operations are simply much cheaper and more deterministic than their contract-implemented equivalents.

5.3 Lifecycle of a Transaction

To make the architecture concrete, consider an SRC-20 transfer of 100 tokens from Alice to Bob.

```
Step 1 — Compose transaction
  Alice's wallet builds:
    op_type    = 1  (SRC-20)
    from       = 0xALICE...
    to         = 0xTOKEN_CONTRACT
    value      = 0
    data       = SRC20_TRANSFER || 0xBOB... || 100
    nonce      = current Alice nonce
    sig        = sign(payload, alice_sk)
    fee        = MIN_TX_FEE = 10,000 sentri

Step 2 — Submit
  Wallet sends to any RPC endpoint.
  RPC validates basic format, places in mempool.
  Mempool gossips tx to peers via libp2p.

Step 3 — Block inclusion
  Validator V (proposer for next round) drains mempool.
  V constructs block N+1 containing Alice's tx + others.
  V proposes block N+1 to active set.

Step 4 — BFT finalization
  Active validators receive proposal, verify it.
  Each validator prevotes if proposal is valid.
  Once ≥⅔ prevotes observed, validators precommit.
  Once ≥⅔ precommits observed, block N+1 is final.

Step 5 — State application
  Each node applies block N+1:
    a. Deduct 10,000 sentri from Alice's account.
    b. Burn 5,000 sentri to BURN_ADDRESS (50% of fee).
    c. Credit 5,000 sentri to validator V's pending rewards (50% of fee).
    d. Decrement Alice's SRC-20 token balance by 100.
    e. Increment Bob's SRC-20 token balance by 100.
    f. Increment Alice's nonce.
    g. Emit Transfer event.
    h. Update state trie root; new root committed in block hash.

Step 6 — Confirmation
  Alice's wallet polls RPC; confirms tx is in finalized block.
  Bob's wallet (subscribed via WebSocket) receives notification.
  Total time from Step 2 to Step 6: ~1–2 seconds.
```

This lifecycle is identical in principle for native transfers, EVM contract calls, and staking operations—the differences are which apply path is taken in step 5.

---

6. Tokenomics

The native asset of Sentrix is SRX. Its monetary policy is fixed.

6.1 Supply

The maximum supply is 315 million SRX. There is no governance mechanism to change this. After the maximum is reached through block rewards, no further SRX is created.

6.2 Block Reward and Halving

The base block reward is one SRX. The reward halves every approximately 126 million blocks (~four years at one-second block time), modeled on Bitcoin's halving cadence. This produces a predictable disinflationary supply curve that converges asymptotically to the cap.

Halving schedule (approximate, four-year cadence):

Era	Years	Block reward	Cumulative issued
1	0–4	1.000 SRX	126M SRX
2	4–8	0.500 SRX	189M SRX
3	8–12	0.250 SRX	220.5M SRX
4	12–16	0.125 SRX	236.25M SRX
5	16–20	0.0625 SRX	244.13M SRX
…	…	…	(asymptotic)

Combined with the 63M premine (Section 6.4), maximum supply approaches 315M SRX over a 24-year horizon, then plateaus.

6.3 Fee Mechanism

Every transaction pays a fee. The fee model differs by execution rail:

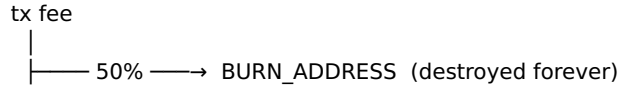
Native rail. A flat MIN_TX_FEE of 10,000 sentri (0.0001 SRX) per transaction, regardless of operation. Native operations have fixed cost at the protocol level.

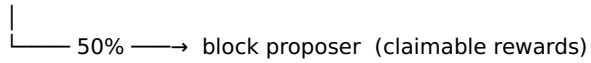
EVM rail. Standard EIP-1559 [13] mechanics: a per-block `baseFeePerGas` that adjusts to congestion, plus a sender-set tip. `gasUsed × (baseFee + tip)` is charged.

Of every fee paid, on both rails:

- **50% is destroyed forever** by sending to a verifiable burn address from which no transaction can be produced. The total burned is publicly observable on chain.
- **50% is paid to the proposer of the block** in which the transaction was included. This forms the variable component of validator rewards on top of the fixed block subsidy.

Figure 3 — Fee split flow





The destruction of half of every fee is the deflationary mechanism. As network activity grows, the rate of destruction grows proportionally, and circulating supply may begin to contract while the fixed block reward contributes new issuance. At sufficient activity, the chain reaches a deflationary equilibrium.

6.4 Premine and Initial Distribution

A premine of 63 million SRX—20% of the supply cap—was allocated at genesis across four roles. All allocation addresses are public and verifiable on chain:

Role	Amount	Address
Founder	21M SRX	0x5b5b06688dcdbe532353ac610aaff41af825279d
Early Validator	10.5M SRX	0x328d56b8174697ef6c9e40e19b7663797e16fa47
Ecosystem Fund	21M SRX	0xeb70fdefd00fdb768dec06c478f450c351499f14
Reserve	10.5M SRX	0x2578cad17e3e56c2970a5b5eab45952439f5ba97

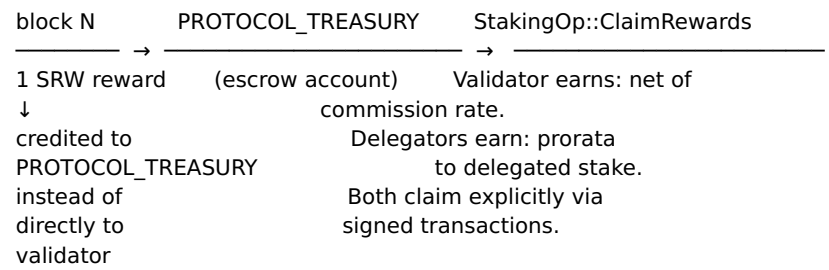
The remaining 80% of supply (252M SRX) issues through block rewards over approximately 24 years, after which no further SRX is issued.

Vesting: The premine has no on-chain vesting schedule. The Founder allocation is operationally treated as a long-term holding by the author—not a tradable position—but this is a behavioral commitment, not a protocol enforcement. Read this paper, audit the addresses, decide your trust accordingly. Future N-of-M multisig migration may impose enforced vesting on a portion of the Founder allocation; until that ships, the allocation is unilaterally controllable.

Treasury management. The Ecosystem Fund (21M SRX) is the primary expansion budget. Intended uses, in priority order: (1) seed liquidity for the first SRX/stablecoin DEX pool when the bridge protocol is live, (2) hackathon prizes and developer grants up to 100K SRX per project, (3) partnership integration subsidies with real-economy counterparties, (4) infrastructure grants for independent validators. Disbursements are visible on chain; the spending pattern over time is the public accountability mechanism.

6.5 Reward Routing

Figure 4 — Reward routing



Block rewards do not pay validators directly. They are credited to a protocol treasury escrow, from which validators and their delegators claim accrued earnings through staking operations. This design preserves the supply invariant—new SRX enters circulation only when claimed, never when produced—and provides a clean accounting boundary between issuance and distribution.

---

7. Validator Economics

Validators are the production layer of Sentrix. Their behavior is guided by three economic forces and one bootstrap procedure.

7.1 Stake at Risk and Slashing

Every validator must bond a minimum self-stake to the protocol. The self-stake is at risk: provable misbehavior results in slashing, the destruction of a portion of bonded stake. The slashing matrix is:

Trigger	Evidence	Stake slashed	Jail duration
Double-sign	Two signatures on conflicting blocks at the same height	20% (parametric)	Permanent (tombstone)
Downtime	Missed > threshold blocks in window	0.1% (parametric)	Configurable jail blocks

The double-sign penalty is severe because the evidence is unambiguous and the act is malicious. The downtime penalty is gentle because legitimate causes (kernel reboot, brief network blip) are common; it scales with persistence rather than punishing transient absence. Both penalties reduce the active stake of the validator and remove them from the active set; re-entry requires explicit unjail and may require additional self-stake top-up.

Slashed SRX is destroyed (added to BURN_ADDRESS), not redistributed. This preserves the supply invariant and avoids creating perverse incentives among non-slashed validators to encourage slashing of competitors.

7.2 Block Reward and Fee Share

Validators earn block rewards (the fixed subsidy) plus fee shares (50% of every transaction fee in blocks they propose). Their delegators earn proportionally to their delegated stake, less a commission set by the

validator.

A validator's expected revenue per epoch is:

```
expected_revenue = (block_subsidy × blocks_proposed)
                 + (fee_share × tx_fees_in_those_blocks)

where:
  blocks_proposed ≈ epoch_length × (validator_stake / total_active_stake)
  fee_share = 0.5
```

Higher-stake validators propose more blocks (proposer rotation is stake-weighted) and earn proportionally more.

7.3 Liveness Penalty

Validators that miss blocks accumulate downtime against a moving window (LIVENESS_WINDOW). Sustained downtime—failing to sign a sufficient fraction of blocks within the window—results in jailing, which removes the validator from the active set and slashes a small portion of stake. Re-entry requires an explicit unjail transaction after a jail duration.

These three forces produce a self-policing economic equilibrium: profitable to validate honestly and reliably; costly to validate maliciously or unreliably.

7.4 Genesis and Bootstrap

Sentrix's genesis state is constructed from a configuration file (`genesis.toml`) declaring:

- The chain ID (7119 mainnet, 7120 testnet).
- Initial premine balances (the 63M SRX allocation across four roles).
- The initial validator set with public keys and self-stake commitments.
- Protocol parameters effective from block 1 (block time, fee constants, fork heights).

Any node can verify the genesis state by re-applying the genesis configuration; the resulting state root must match the hardcoded genesis hash. New nodes joining mainnet bootstrap by connecting to seed peers, syncing block-by-block from genesis (or from a recent state snapshot signed by trusted validators), and entering the active validator set when their stake-weighted ranking enters the top-K threshold for the next epoch.

7.5 Becoming a Validator

The validator set is open and permissionless. Any operator who can run reliable infrastructure and bond minimum self-stake may register.

Concrete requirements:

- **Hardware:** A modern x86-64 server with ≥4 CPU cores, ≥8 GB RAM, ≥250 GB SSD storage, and stable network connectivity. A single virtual private server in a reputable datacenter is sufficient for the present scale; production validators typically run on 8 GB RAM with comfortable headroom.
- **Network:** A stable public IPv4 address with TCP port 30303 reachable for the libp2p P2P transport.
- **Self-stake:** A minimum bond of native SRX tokens, locked while active and during the unbonding period. The exact threshold is a protocol parameter (see Appendix A).
- **Identity:** A registered validator wallet (secp256k1 keypair) with a

human-readable validator name. The wallet signs blocks and votes; secure key management is the operator's responsibility.

Registration flow: operator runs `sentrix validator register` with their wallet keystore, paying the registration transaction fee + bonding the minimum self-stake. Once registered, the validator is eligible for active-set inclusion; whether they enter the active set in the next epoch depends on stake-weighted ranking. Delegators can begin delegating to the validator immediately after registration.

There are no jurisdictional restrictions, no whitelist, no application process. Misbehavior is enforced economically (slashing) rather than administratively. We treat the open-validator-set property as a foundational decentralization guarantee.

7.6 Incident Response

A live blockchain occasionally faces consensus stalls, software bugs, network partitions, or coordinated attacks. Sentrix's incident response model is structured around three principles:

Detection through observation. Every full node independently verifies block applications. State divergence produces divergent block hashes; nodes with divergent state cannot win the canonical chain. A `sentrix-watchdog` daemon runs against the public RPC and pages the operator on stall (no block advance for >5 minutes) or per-validator divergence detection.

Recovery through canonical-state alignment. When validators diverge—for example, after a hard fork is mis-applied at one node, or after a node's chain.db is corrupted—the recovery protocol is to identify the canonical state (the chain hash that supermajority of stake-weighted validators agrees on) and replicate that chain.db to the diverged node. This is a well-defined, repeatable operational procedure documented in operator runbooks.

Coordinated upgrade through hard fork. Bug fixes that require consensus changes ship as hard forks gated by activation height. Operators upgrade their binary before the activation height; on activation, all nodes apply the new logic atomically. The window between binary release and activation height (typically 1–7 days for non-urgent fixes, hours for urgent ones) is the coordination period during which the network agrees on the upgrade.

The chain has no on-chain governance veto in its current state; protocol upgrades are coordinated by binary releases. As the chain matures and decentralization deepens (Stage 5 in the path forward), this transitions to formal on-chain governance.

---

8. Security Model

Sentrix's security derives from four layers of guarantee.

8.1 Cryptographic Guarantee

Every transaction is signed with a private key whose public address authorizes the operation. Block headers contain the state root and the proposer's signature. The state root commits to the canonical state via the binary sparse Merkle tree; a validator that applies the same blocks reaches the same state and signs the same root.

8.2 Crypto-Economic Guarantee

Validators participate because it is profitable to do so honestly. They refuse to participate dishonestly because slashing makes dishonest validation negative-expected-value at any stake size. The slashing rates are calibrated to make a successful safety-violating attack require a coordinated stake commitment of at least one-third of the total—a stake commitment whose marginal cost (X·P/3 where X is total bonded SRX and P is the SRX market price) grows with chain success.

8.3 Social Guarantee

The chain's behavior is observable. State, blocks, transactions, validator set, slashing events—all are public. Operators of full nodes can independently verify every transition. Auditors can reconstruct any historical state. The codebase is source-available under a license that becomes fully open after a defined Change Date, so the chain's behavior is verifiable at the source level. Trust is replaced by verification.

8.4 Quantitative Security

The cost of compromising chain safety is bounded below by the cost of acquiring and slashing one-third of the total bonded SRX. If the protocol bonds X SRX at market price P, then the minimum attack cost A satisfies:

$$A \geq (X \times P) / 3$$

This is a lower bound, not an upper bound: practical attacks face additional costs (coordinating stake acquisition without market disturbance, executing the attack within the unbonding window, accepting permanent reputational damage). The bound rises monotonically with both X (bonded supply) and P (market price), which means chain success increases attack cost.

For a worked example: if 100M SRX are bonded at a market price of $0.10/SRX, the minimum attack cost is approximately $3.3M. At 200M bonded and $1.00/SRX, the cost rises to ~$67M. At 300M bonded and $5.00/SRX, the cost is ~$500M. These figures are illustrative; actual values depend on market dynamics.

8.5 Long-Range Attacks and Weak Subjectivity

Pure proof-of-stake chains are theoretically vulnerable to long-range attacks: an attacker who acquires old validator keys (after their owners have unbonded and sold them) can fork the chain from an arbitrarily early point. Sentrix defends against this through three mechanisms:

- **Unbonding period.** Stake is slashable for a fixed duration after withdrawal. An attacker acquiring keys must forge blocks before the unbonding period elapses, or the keys are no longer slashable.
- **Weak subjectivity checkpoints.** New nodes joining the network are expected to start from a recent block hash they trust through some out-of-band channel (a peer, a trusted validator, the project website). This bounds how far back an attacker's fork can plausibly be presented as canonical.
- **Periodic active-set sync.** Light clients re-sync their trusted validator set at intervals shorter than the unbonding period. This prevents an attacker who has acquired many old keys from presenting a self-signed alternative chain as the canonical one to a long-offline client.

These mechanisms are standard for BFT proof-of-stake chains [14]. They do not eliminate the threat in theory but reduce it to a model in which any honest reference point within the unbonding period prevents successful long-range attack.

8.6 Failure Modes

The chain has four well-defined failure modes:

- **More than ⅓ Byzantine stake.** Safety is no longer guaranteed. The chain may fork. Recovery requires social coordination to identify the canonical chain.
- **More than ⅔ Byzantine stake.** Invalid state can be produced and signed. Detection requires honest full nodes verifying state independently; once detected, recovery is by social coordination.
- **Network partition.** A minority partition halts because it cannot reach supermajority. A majority partition continues with reduced active set. When the partition heals, the minority detects the longer canonical chain and rejoins.
- **Coordinated censorship of specific transactions.** Resistant to one-time censorship because proposer rotation guarantees that any non-censoring active validator will eventually propose a block. Sustained censorship requires control of the active validator set, which by stake-weighted selection requires majority stake control.

The model assumes no failure modes outside this set are silent: any deviation from honest behavior produces evidence visible to other full nodes, slashable to validators, and observable to the public.

8.7 Privacy Posture

Sentrix is **transparent by design**. Every transaction, balance, and contract call is publicly observable. This is a deliberate choice: the chain serves real-economy settlement, where audit trails are a feature rather than a liability. We do not attempt to provide built-in privacy primitives (zk-shielded transactions, anonymous balances, mixers).

Users who require privacy for specific use cases can build it on top: zk-rollup contracts, privacy-preserving DApps, off-chain commitments verified on-chain. The base layer remains observable; privacy is opt-in at the application layer.

This design choice has tradeoffs. Transparent chains are easier to audit, easier to integrate with regulatory frameworks, and easier to reason about at the protocol level. They are less suitable for operators who require mandatory privacy (witness protection, victim of harassment, sensitive commercial counterparties). Sentrix accepts the tradeoff in favor of regulatory legibility.

---

9. The Real-Economy Thesis

Sentrix’s positioning is real-world settlement. We expand on what this means and why we believe it represents a meaningful and unmet demand.

Real-world assets. Tangible and contractual rights—real estate, invoices, equity in private companies, rights to revenue streams, bills of lading—are valuable to their holders. They are also illiquid: difficult to fractionalize, slow to transfer, expensive to verify. A blockchain that can represent these as on-chain tokens, with provable provenance and atomic settlement, removes friction proportional to the value of the asset. Sentrix is designed to make this representation cheap and fast.

Local-currency settlement. Most economic activity is denominated in local currencies—rupiah, peso, dong, baht, ringgit, dirham. A blockchain that supports stablecoin issuance against these currencies, with native operations for cheap fast transfer, becomes a settlement rail for retail commerce that does not currently have one. Sentrix's native SRC-20 operations are designed to make local-stablecoin issuance and use approximately as cheap as a SWIFT transaction is expensive.

Cross-border commerce and remittance. Cross-border payments traditionally settle through correspondent banking with multi-day delays and fees that disproportionately burden smaller transactions. A blockchain whose native operations cost a fraction of a cent and finalize in one second is fundamentally suited to small-ticket cross-border flow. Indonesia is a remittance-receiving country at the scale of tens of billions of dollars annually; the unmet efficiency demand is significant.

Microfinance and cooperative settlement. Group savings (arisan), cooperative banking (koperasi simpan-pinjam), and community lending are deeply embedded in Indonesian economic culture. They operate on trust, paper records, and informal coordination. A blockchain that can represent these flows—on-chain, verifiable, low-cost—is an upgrade rather than a replacement.

We do not claim Sentrix solves these. We claim its architecture is shaped to be useful for them, where chains optimized for trading speculation are not.

---

10. Comparison to Prior Work

Sentrix sits in a lineage of public blockchains whose design choices we acknowledge and contrast.

	Block time	Finality	Consensus	VM	Supp poli
Bitcoin [1]	~10 min	Probabilistic	Proof of Work	None	Cappe halving
Ethereum [2]	~12 sec	Probabilistic → final	Proof of Stake	EVM	Inflatio (variab
Solana [5]	~400 ms	Probabilistic	PoH + TowerBFT	SVM (BPF)	Inflatio (declin
Cosmos Hub [3,6]	~6 sec	Single-block	Tendermint BFT	Cosmos SDK (Go modules)	Inflatio
Polygon	~2 sec	Probabilistic + checkpointed	PoS BFT variant	EVM	Inflatio (cappe
Aptos	~250 ms	Single-block	AptosBFT	Move	Inflatio
Sui	~390 ms	Single-block (per-object)	Mysticeti BFT	Move (object-centric)	Inflatio
Near	~1.2 sec	Single-block	Nightshade (sharded)	NEAR VM (Wasm)	Inflatio
Sentrix	**~1 sec**	**Single-block**	**DPoS + BFT (Tendermint-like)**	**EVM (revm) + Native rail**	**Cappe halvin 50% b**

Bitcoin [1] established the proof-of-work secured public ledger. Sentrix borrows the supply discipline (capped, halving) but rejects proof-of-work as an energy commitment we are unwilling to make. We use stake-weighted Byzantine Fault Tolerant consensus instead, sacrificing the permissionless mining property in exchange for one-second finality and approximately zero energy cost per block.

Ethereum [2] established programmable smart contracts as a dominant paradigm. Sentrix preserves EVM compatibility precisely because we do not wish to reinvent its developer ecosystem. We diverge on the native-versus-contract question: Ethereum chose maximum uniformity (every primitive is a contract); Sentrix chooses pragmatic specialization (common primitives are native).

Solana [5] demonstrated that high-throughput single-shard chains are viable at low validator counts. Sentrix targets a similar performance envelope but takes a different consensus path (BFT with explicit precommits versus Solana's Proof of History timestamping). Sentrix's design also maintains EVM compatibility, which Solana does not.

Cosmos [3] [6] established the Tendermint BFT family and the inter-blockchain communication standards we draw from. Sentrix's consensus is closer to the Tendermint family than to any other lineage; we differ in our choice of EVM compatibility over the Cosmos SDK execution model.

Polygon, BNB Chain, Avalanche C-Chain. Various chains have built EVM-compatible high-performance environments. Each has chosen a different positioning within the speculative-DeFi space. Sentrix's distinct positioning is the real economy and the Indonesia-first market entry.

Aptos, Sui, Near. Newer chains with novel execution models—Move (object-centric and resource-typed) and Wasm. These offer attractive correctness properties for new contracts written ground-up. Sentrix's choice of EVM compatibility is deliberate: existing Solidity dApps deploy unchanged, the developer ecosystem already exists at scale, and tooling (Foundry, Hardhat, MetaMask, ethers, viem) is mature. Move and Wasm chains require greenfield rewriting of every dApp—a steep adoption tax for a new chain to ask developers to pay.

Sentrix is not novel in any single dimension. Its contribution is the combination: native-EVM hybrid execution, stake-weighted BFT, deflationary capped supply, sub-second finality, oriented toward real-world settlement and rooted in the Indonesian market.

---

11. Governance

11.1 Current State

Sentrix's protocol upgrades are coordinated through binary releases gated by activation height. The author releases new node binaries; operators upgrade in the time window before a fork's activation height; on activation, all nodes apply the new logic atomically. This is the same mechanism Bitcoin, Ethereum, and most Tendermint-based chains use for hard forks.

In the current single-author phase, the author holds an effective veto over what releases ship. This is appropriate for early-stage development—rapid iteration, fast bug response, no committee bottleneck—but is not the long-term governance model.

11.2 The SentrixSafe Multisig

Authority over privileged operations (validator authority key, treasury reserve management, fee-fork toggle) is held by the SentrixSafe multisig, a Gnosis-Safe-derived contract deployed at genesis. Currently configured 1-of-1 with the author as sole signer; intended to expand organically to N-of-M as the chain attracts long-term contributors who can credibly sign off on protocol authority operations.

Expansion happens by adding co-signers through SentrixSafe's standard `addOwner` operation, increasing the signature threshold proportionally. There is no hard timeline; the bar is "credible co-signer with operational continuity and skin-in-the-game," not "calendar quarter."

11.3 Future On-Chain Governance

Stage 5 of the path forward (Section 12) migrates protocol decisions onto a stake-weighted on-chain governance mechanism. The expected design:

- **Proposal threshold:** Anyone holding above a minimum SRX stake may submit a proposal.
- **Voting:** Stake-weighted vote across the active validator set, with delegators inheriting their validator's vote unless they explicitly override.
- **Quorum:** Proposals require minimum participation (target: 33% of active stake) to be valid.
- **Pass threshold:** Configurable per proposal type—simple majority for most decisions, supermajority for protocol-breaking changes.
- **Execution:** Passing proposals trigger pre-defined on-chain effects (parameter updates, treasury disbursements, hard-fork activation height set).

Until Stage 5 ships, the governance discipline is operational: privileged operations only via SentrixSafe, transparent treasury usage, public commit history, and binary releases coordinated openly.

11.4 What Cannot Be Governed

Some chain properties are deliberately non-governable:

- The 315M SRX supply cap. No vote, no fork, no upgrade changes this. The cap is part of Sentrix's foundational economic contract.
- The four-year halving schedule. Locked into block-reward calculation.
- The 50% fee burn ratio. Locked into fee dispatch.
- The genesis allocation (63M premine across the four roles). Allocated at block 0; no mechanism exists to undo it.

These are the parameters where stability is the feature. Everything else (network parameters, validator-set size, fork heights for new opcodes) is subject to coordinated upgrade.

---

12. Path Forward

We describe the trajectory of Sentrix in the language of stages rather than dates, because dates create false specificity that real conditions never honor.

Stage 1: Mainnet operating. A small, geographically distributed validator set. Native and EVM execution working correctly. Block explorer, faucet, dev tooling. Source-available codebase. Reachable from standard Ethereum tooling. *(Current state.)*

Stage 2: Liquidity and discovery. First on-chain market for SRX. First contracts deployed by external developers. Indexing in standard chain registries. Recognition by standard wallets and bridges. Initial community.

Stage 3: Real-economy integrations. First production use cases involving real-world assets, local-currency settlement, or cross-border flow. The specifics emerge from market discovery; we will not predict them in advance.

Stage 4: Validator decentralization. Growth of the validator set from a small bootstrap to a meaningful number of independent operators across multiple jurisdictions. Open delegation. Mature slashing economics.

Stage 5: On-chain governance. Migration of meaningful protocol decisions onto a stake-weighted governance mechanism. The foundation's role narrows.

Stage 6: Open license activation. The codebase transitions from source-available to fully open source under standard terms. Anyone may fork the chain or run an alternative network derived from its code.

These stages do not run on a calendar. They run on the satisfaction of preconditions. We will not announce dates we cannot guarantee.

---

13. Conclusion

Sentrix is a deliberate response to a structural absence. The dominant blockchains of the present moment serve trading and speculation well, and they serve real economic settlement poorly. We do not believe this is inevitable. We believe it is the consequence of design choices that can be made differently.

Sentrix's design choices are: native operations for common primitives, EVM compatibility for the general case, stake-weighted Byzantine Fault Tolerant consensus, sub-second finality, capped halving deflationary supply, and a market-entry orientation toward the Indonesian economy and the broader unmet demand for real-world settlement infrastructure.

We expect to be in operation for a long time. The economic forces shaping participation in public blockchains are pre-political and pre-narrative; they will continue to apply long after this paper is no longer being read. We have shaped the chain such that those forces work in favor of its longevity, and against the kind of speculative collapse that has consumed projects whose only economic content was speculation.

Sentrix is open to use, open to extension, and open to inspection. Real value, real assets, real economic activity—on chain, fast, final, and durable.

Appendix A — Protocol Parameters

Parameter	Value	Notes
BLOCK_TIME	~1 s	Target; actual varies with round duration
MAX_TX_PER_BLOCK	5,000	Configurable at protocol level
MAX_SUPPLY	315,000,000 SRX	Hard cap, no governance override

INITIAL_BLOCK_REWARD	1 SRX	Halves every HALVING_PERIOD
HALVING_PERIOD_BLOCKS	~126,000,000	~4 years at 1 s blocks
MIN_TX_FEE	10,000 sentri	0.0001 SRX (native rail)
BURN_RATIO	50%	Of every transaction fee
MIN_VALIDATOR_SELF_STAKE	parametric	Configured in genesis
UNBONDING_PERIOD	parametric	Stake-slashable window after withdrawal
LIVENESS_WINDOW	14,400 blocks	~4 hours at 1 s blocks
MIN_SIGNED_PER_WINDOW	4,320 blocks	30% of LIVENESS_WINDOW
JAIL_DURATION_BLOCKS	600	~10 minutes at 1 s blocks
DOWNTIME_SLASH_BP	10 (0.1%)	Of self-stake on jail
EPOCH_LENGTH	parametric	Active-set rotation cadence
STATE_ROOT_FORK_HEIGHT	activated	State root committed in block hash

Parameters marked “parametric” are configurable at deploy time and may be tuned through governance after Stage 5 (Section 11).

References

[1] Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System.*

[2] Buterin, V. (2014). *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform.*

[3] Buchman, E., Kwon, J. (2016). *Tendermint: Byzantine Fault Tolerance in the Age of Blockchains.*

[4] Bluealloy (2022). *revm: Rust Ethereum Virtual Machine.*

[5] Yakovenko, A. (2017). *Solana: A new architecture for a high performance blockchain.*

[6] Kwon, J., Buchman, E. (2019). *Cosmos: A Network of Distributed Ledgers.*

[7] Fischer, M., Lynch, N., Paterson, M. (1985). *Impossibility of Distributed Consensus with One Faulty Process.*

[8] Castro, M., Liskov, B. (1999). *Practical Byzantine Fault Tolerance.*

[9] Merkle, R. (1980). *Protocols for Public Key Cryptosystems.*

[10] MDBX. *Memory-mapped key-value store.* https://github.com/erthink/libmdbx

[11] libp2p. *Modular peer-to-peer networking stack.* https://libp2p.io

[12] Maymounkov, P., Mazières, D. (2002). *Kademlia: A Peer-to-peer Information System Based on the XOR Metric.*

[13] Buterin, V. et al. (2019). *EIP-1559: Fee market change for ETH 1.0 chain.*

[14] Buterin, V., Griffith, V. (2017). *Casper the Friendly Finality Gadget.*

[15] Dwork, C., Lynch, N., Stockmeyer, L. (1988). *Consensus in the Presence of Partial Synchrony.*

Appendix B — Risk Disclosures

This appendix documents known risks. It is not exhaustive; readers must perform their own due diligence.

Technical risks. - *Consensus non-determinism class.* The chain has experienced LivenessTracker non-determinism halts that required disabling the consensus-jail dispatch (`JAIL_CONSENSUS_HEIGHT=u64::MAX`). Manual jailing remains operational. A permanent fix is in scope for future fresh-brain development sessions. - *Single-implementation risk.* The chain has one Rust implementation. A bug in that implementation can affect the entire network until patched. Multiple-implementation diversity (Bitcoin's Bitcoin Core / btcd / Knots model) is not present. - *Validator concentration.* The active validator set is small at this stage. Sustained Byzantine behavior by a supermajority is theoretically possible until the active set grows to a meaningfully decentralized size.

Economic risks. - *No price discovery yet.* SRX is not currently trading on any exchange or DEX. There is no market price. The 315M cap is a protocol constant, not a market valuation. - *Founder allocation is unilaterally controllable.* Until SentrixSafe expands to N-of-M and/or Founder allocation is on-chain vested, the Founder address can move 21M SRX at any time. - *Stablecoin/bridge dependency.* Real-economy use cases require a stablecoin counterparty. Until a bridge protocol is deployed, Sentrix is islanded from the broader stablecoin economy.

Regulatory risks. - *Indonesian regulatory landscape evolving.* Bappebti (Indonesia commodity futures regulator) has framework for crypto assets that continues to evolve. Sentrix operates in this evolving environment and may face new requirements. - *Cross-jurisdictional uncertainty.* Holders and validators in different jurisdictions face different regulatory regimes that may classify SRX or staking activity differently. Consult local counsel. - *Securities-law classification.* SRX is intended as a utility token for chain operations (gas, staking, governance). Whether any specific jurisdiction classifies it as a security depends on local law and how it is offered. The author is not legal counsel.

Operational risks. - *Single-author bus factor.* Sentrix is solo-built. The author's continued participation is not guaranteed by any contract. Long-term resilience depends on the codebase being open enough that other operators can run forks (BUSL → Apache 2.0 transition after Change Date). - *Infrastructure single points of failure.* The validator hosts, the explorer host, the DNS configuration, the documentation site—all are operationally maintained by the author at this stage. Decentralization of these operational layers is a forward objective, not a current state.

This list is honest, not exhaustive. The chain's behavior, contract, and history are public; readers should verify what they care about against the on-chain record and the source code.

Appendix C — Legal Notice

This whitepaper is a description of a software protocol. It is not an offering document, prospectus, investment solicitation, or financial advice. SRX is a utility token used to pay transaction fees, secure the chain through staking, and (in the future) participate in governance. Acquiring or holding SRX entails risks—technical, economic, regulatory, and operational—described in Appendix B.

The Sentrix Chain protocol is open-source infrastructure under the Business Source License 1.1, transitioning to Apache 2.0 after the Change Date specified in the LICENSE file. Anyone may run a node, anyone may submit transactions, anyone may operate as a validator subject to the protocol-level requirements described in §7.5.

The author makes no representations about future SRX market price, ecosystem adoption, partnership outcomes, or regulatory acceptance. Forward-looking statements in §12 (Path Forward) describe intent, not guarantees.

Readers are responsible for compliance with their local laws and regulations regarding cryptocurrency holding, transaction, and validator operation.

About the Author

Satya Kwok built Sentrix Chain solo in Rust. The author's prior work and engagement is publicly observable on GitHub (@satyakwok) and through the project's open commit history at github.com/sentrix-labs/sentrix. The author is contactable through the channels listed at sentrixchain.com and through the issue tracker on the canonical repository.

The decision to build Sentrix solo was deliberate: small teams ship faster, decisions are clearer, and accountability is unambiguous. The trade-off is the author's bus factor (Appendix B). The author is committed to operating Sentrix as durable financial infrastructure for the indefinite future, but commits to no specific timeline beyond the protocol-level guarantees codified in the chain itself.

Acknowledgments

Sentrix is the product of a single author's work over an intentionally compressed period. The author acknowledges the engineers who built the foundations on which Sentrix is constructed—the Ethereum core developers, the Cosmos and Tendermint teams, the Bitcoin community, the Rust ecosystem, the open-source maintainers of the cryptographic and networking libraries that make a project of this scope feasible for a single individual to undertake.

The decision to build Sentrix in Rust, on the EVM, with Tendermint-style BFT consensus, did not require inventing any of these components. It required only their composition. This is how durable infrastructure is built: not from new ideas, but from the careful arrangement of existing ones.